AI-POWERED GITHUB REPOSITORY

INTELLIGENCE & CODE REVIEW

Project documentation | Demonstration and submission copy

Prepared by: _____

Project: Repo Intel

1. Project Definition

Repo Intel is a web-based software platform that analyzes GitHub repositories and produces a structured, evidence-based code review. A user submits a repository URL; the platform inspects eligible source files and returns an overall score, category scores, technology and architecture summaries, and actionable findings with file and line references.

The system combines deterministic checks with an optional large-language-model review. Users can also upload coding guidelines so the AI reviewer can compare supplied code samples against their guidance. The platform is intended as a development aid and learning tool, not as a replacement for a full security audit or human review.

2. Problem and Objectives

- Reduce the effort required to understand an unfamiliar repository and identify common quality, security, performance, and best-practice concerns.
- Provide explainable findings: each issue includes code evidence, location, reason, severity, and a recommended action.
- Summarize repository technologies and architecture in one report, with analysis history available to the signed-in user.
- Support privacy-aware use of uploaded guidelines and optional GitHub authorization for private repositories.

3. Main User Features

Account registration and sign-in; email verification and password recovery; repository analysis; report history; guideline upload and management; and private-repository authorization when configured.

4. System Architecture

The application is split into a browser-facing Django web application and a separate FastAPI analysis service. Django handles users, pages, saved analyses, and report display. The AI service retrieves repository content from GitHub, runs the analysis workflow, and returns report data to Django over an internal authenticated API.

REQUEST AND RESULT FLOW

Browser -> Django web app -> FastAPI analysis service

-> GitHub + review workflow -> report JSON

Django stores the completed analysis and displays the report to its owner.

5. Technology Stack

Frontend / UI	Django templates, HTML, CSS, and JavaScript. The browser UI includes authentication, dashboard, analysis forms, reports, history, and guideline management.
Web backend	Python and Django 5.x. Handles account workflows, application pages, authorization, analysis polling, and persistence.
AI service	Python, FastAPI, LangGraph, and LangChain. Coordinates repository analysis and produces structured review output.
AI / language model	Groq API with a configurable model for AI-assisted review and summaries. Rule-based checks provide a fallback when AI is unavailable.
Primary database	PostgreSQL 16 in the Docker Compose deployment. SQLite is used as a lightweight local-development fallback when PostgreSQL settings are absent.
Guideline store	Chroma vector database for indexed, user-scoped guideline passages used in retrieval-augmented review.
External integration	GitHub API for repository access; optional GitHub OAuth for private repositories. Docker Compose runs the web, AI, and database services.

6. High-Level Project Structure

ai_service/	FastAPI application, analysis workflow, GitHub client, LLM integration, RAG, tests
web/accounts/	Django account, registration, verification, and password workflows
web/reviews/	Analysis, reports, findings, guideline management, and related tests
web/templates/ + web/static/	HTML templates, styles, and browser-side assets
web/config/	Django project configuration and URL routing
docker-compose.yml	Local multi-service orchestration for PostgreSQL, web, and AI service

7. Analysis Workflow

1. Submit

The signed-in user enters a GitHub repository URL and chooses a public or authorized private-repository flow.

2. Fetch and detect

The analysis service validates the repository reference, retrieves eligible repository content, and identifies languages, frameworks, and other technologies.

3. Select and protect

Source files are prioritized and split into chunks. Secrets are masked before code is sent to an AI provider; environment files are not read as source.

4. Review

Static checks run alongside a structured AI review for relevant code samples. Uploaded guideline passages can be included in that review.

5. Verify and score

Quoted evidence is checked against source content, findings are grouped and scored, and the service prepares architecture and summary information.

6. Save and display

The completed report is returned to Django, stored with the user account, and shown in the report and analysis history pages.

8. Security, Privacy, and Limitations

- Analysis and guideline records are scoped to the signed-in user. Private repository access uses a GitHub authorization flow when configured.
- The AI review samples selected files and code chunks to manage request size and cost; it does not guarantee that every line or every issue is reviewed.
- AI findings can be incomplete or incorrect. Evidence verification reduces unsupported findings, but users should validate recommendations before making changes.
- Repository access can be constrained by GitHub API rate limits. AI-provider limits and request-size limits can also affect an analysis.
- The platform is a static review aid, not a penetration test. Uploaded guideline coverage depends on whether relevant code is included in the reviewed sample.

9. Future Enhancements

- Expand code coverage while preserving clear context and manageable analysis costs.
- Add more automated security and quality tools, richer test coverage, and improved queueing for long-running jobs.
- Provide export options and clearer per-file visibility into which code and guidelines were evaluated.

CONCLUSION

Repo Intel brings repository discovery, evidence-backed review, and actionable guidance together in a single web-based workflow for developers and project teams.